

AUTOM-03: Monaco Configuration-as-Code

Series: AUTOM — Dynatrace Automation | **Notebook:** 3 of 9 | **Created:** January 2026 | **Last Updated:** 09/18/2026

Monaco (Monitoring as Code) is Dynatrace's official CLI tool for configuration management. It uses YAML files to define configurations and supports version control, CI/CD integration, and multi-environment deployments.

Table of Contents

- [1. Introduction](#)
 - [2. Getting Started](#)
 - [3. Project Structure](#)
 - [4. Configuration Files](#)
 - [5. Common Operations](#)
 - [6. Advanced Features](#)
 - [7. Next Steps](#)
-

Prerequisites

Before starting this notebook, ensure you have:

Requirement	Description
Monaco CLI	Installed from the GitHub releases binary (Dynatrace publishes no Homebrew formula)

Requirement	Description
API Token	Token with <code>settings.read</code> , <code>settings.write</code> , <code>ReadConfig</code> , <code>WriteConfig</code>
Tenant URL	Your Dynatrace SaaS tenant URL

Learning Objectives

By the end of this notebook, you will:

- Understand Monaco's configuration-as-code model
- Know how to download existing configurations
- Be able to create and deploy configurations via YAML
- Handle multi-environment deployments

1. Introduction

Why Monaco?

Benefit	Description
Version Control	Track all changes in Git
Repeatability	Deploy same config to multiple environments
Review Process	Use pull requests for config changes
Disaster Recovery	Quickly restore configurations
Documentation	YAML files serve as living documentation

Monaco vs Direct API

Aspect	Monaco	Settings API
Format	YAML files	JSON payloads
State management	Implicit (by name)	Manual tracking
Multi-tenant	Built-in support	Custom scripting
Dry run	Yes (<code>--dry-run</code>)	No
Delete orphans	Yes (<code>delete</code> command)	Manual

Already a Terraform shop? See **AUTOM-01 §5: When a Terraform Shop Should Add Monaco** for the four patterns where Monaco still earns its place — most commonly bulk download from an existing tenant, multi-tenant identical-config deployment, and app-team self-service without state management.

2. Getting Started

Installation

Monaco ships as a single binary on GitHub releases; there is no Homebrew formula. Pick the asset for your OS and architecture (the install guide also lists `monaco-darwin-amd64` , `monaco-linux-arm64` , `monaco-linux-386` and a SHA-256 file for each):

macOS (Apple silicon):

```
curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-darwin-arm64 -o monaco
chmod +x monaco
```

Linux (x86-64):

```
curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-linux-amd64 -o monaco
chmod +x monaco
```

Windows: download the `.exe` asset from the same releases page.

Verify installation:

```
monaco version
```

Environment Setup

Create environment variables for authentication:

```
export DT_TENANT_URL="https://{tenant-id}.live.dynatrace.com"
export DT_API_TOKEN="&lt;your-api-token&gt;"
```

Monaco never takes a secret on the command line. Both the manifest and the `--token` download flag take the **name** of the environment variable that holds the token (`DT_API_TOKEN`), not its value.

Your First Download

Download all configurations from your tenant:

```
monaco download \
  --url "$DT_TENANT_URL" \
  --token DT_API_TOKEN \
  --output-folder ./downloaded-config
```

`--token` names the environment variable (no `$`). An access token covers settings and classic configuration APIs; to also download platform configurations (workflows, documents, buckets, segments), add `--platform-token <VAR_NAME>` or the `--oauth-client-id / --oauth-client-secret` pair — access tokens and platform tokens are not interchangeable.

Download specific Settings 2.0 schemas (`--settings-schema`; `--api` is for **classic** Configuration APIs only):

```
monaco download \
  --url "$DT_TENANT_URL" \
  --token DT_API_TOKEN \
  --output-folder ./downloaded-config \
  --settings-schema builtin:management-zones,builtin:tags.auto-tagging
```

Downloading classic schemas is the point here, not an oversight. Both schemas above are marked **Blocked at upgrade** in AUTOM-02's Settings 2.0 catalog — they work today and stop answering once your tenant moves to the latest Dynatrace. That makes `monaco download` the fastest way to *inventory* what you still have before migrating it: every object, in version control, in one command. Keep the export as your migration worksheet; retarget the deployment. What it should not become is a template for new configuration.

3. Project Structure

Recommended Layout

```
dynatrace-config/  
├─ manifest.yaml          # Project manifest  
├─ environments/  
│   ├─ development.yaml  # Dev environment config  
│   ├─ staging.yaml       # Staging config  
│   └─ production.yaml   # Production config  
└─ projects/  
    └─ my-project/  
        ├─ management-zones/  
        │   └─ config.yaml  
        ├─ auto-tagging/  
        │   └─ config.yaml  
        └─ alerting-profiles/  
            └─ config.yaml
```

Manifest File

The `manifest.yaml` defines your project:

```

manifestVersion: 1.0

projects:
  - name: my-project
    path: projects/my-project

environmentGroups:
  - name: default
    environments:
      - name: development
        url:
          type: environment
          value: DT_DEV_URL
        auth:
          token:
            name: DT_DEV_TOKEN
      - name: production
        url:
          type: environment
          value: DT_PROD_URL
        auth:
          token:
            name: DT_PROD_TOKEN

```

`url` accepts `type: environment + value: <VAR_NAME>`, but every `auth` entry takes only `name: <VAR_NAME>` — Monaco always loads secrets from environment variables. Add `platformToken: {name: DT_PLATFORM_TOKEN}` (Monaco 2.24.0+) or an `oAuth` client alongside `token` if the project also manages workflows, documents, buckets or segments.

4. Configuration Files

About the schemas used in these examples. Management zones, auto-tagging and alerting profiles appear throughout §4–§6 because they are the configurations most tenants already have, which makes them the clearest illustrations of Monaco's YAML shape. All three are marked **Blocked at upgrade** in AUTOM-02's Settings 2.0 catalog, and all three are on Dynatrace's published [Settings 2.0 schemas that are removed in Latest Dynatrace \(DT docs\)](#) — *"None of the schemas on this page are visible in Latest Dynatrace."*

That matters more for Monaco than for the raw API. Monaco resolves a schema before writing objects, so once a schema is removed, `monaco deploy` fails for every config that references it — and it fails at deploy time, in your pipeline, not at edit time. Treat these as **Monaco mechanics you can apply to any schema**, and check a schema's upgrade status before building a long-lived project around it.

The `type.settings.schema` block is schema-agnostic: swap the schema string and the matching template JSON and everything else here is unchanged. Carry-forward schemas worth practising on include `builtin:davis.anomaly-detectors`, `builtin:anomaly-detection.services`, and the `builtin:openpipeline.<scope>.*` family.

Basic Configuration Structure

```
configs:
  - id: production-zone
    config:
      name: Production
      template: management-zone.json
    type:
      settings:
        schema: builtin:management-zones
        scope: environment
```

Management Zone Example

config.yaml:

```
configs:
  - id: production-mz
    config:
      name: Production
      template: production-mz.json
    type:
      settings:
        schema: builtin:management-zones
        scope: environment

  - id: staging-mz
    config:
      name: Staging
      template: staging-mz.json
    type:
      settings:
        schema: builtin:management-zones
        scope: environment
```

production-mz.json:

```
{
  "name": "{{ .name }}",
  "rules": [
    {
      "type": "SERVICE",
      "enabled": true,
      "conditions": [
        {
          "key": {
            "type": "STATIC",
            "attribute": "SERVICE_TAGS"
          },
          "comparisonInfo": {
            "type": "TAG",
            "operator": "EQUALS",
            "value": {
              "context": "CONTEXTLESS",
              "key": "environment",
              "value": "production"
            },
            "negate": false
          }
        }
      ]
    }
  ]
}
```

Auto-Tagging Example

config.yaml:

```
configs:
- id: application-tag
  config:
    name: Application
    template: application-tag.json
  type:
    settings:
      schema: builtin:tags.auto-tagging
      scope: environment
```

application-tag.json:

```
{
  "name": "{{ .name }}",
  "rules": [
    {
      "type": "SERVICE",
      "enabled": true,
      "valueFormat": "{Service:DetectedName}",
      "propagationTypes": ["SERVICE_TO_PROCESS_GROUP_LIKE"],
      "conditions": []
    }
  ]
}
```

Using Variables

Monaco v2 reads environment variables through **environment parameters**, declared under `parameters:` with a `name` (the variable) and an optional `default`. The template then refers to the parameter by its key — there is no `{{ .Env.X }}` syntax in v2:

config.yaml:

```
configs:
- id: alerting-profile
  config:
    name: Alerting
    template: alerting.json
    parameters:
      environment_name:
        type: environment
        name: ENVIRONMENT_NAME
      severity_level:
        type: environment
        name: ALERT_SEVERITY
        default: ERRORS
    type:
      settings:
        schema: builtin:alerting.profile
        scope: environment
```

In `alerting.json`, use `{{ .environment_name }}` and `{{ .severity_level }}`. Without a `default`, the deployment fails if the variable is unset.

5. Common Operations

Deploy Configurations

Deploy to all environments:

```
monaco deploy manifest.yaml
```

Deploy to specific environment:

```
monaco deploy manifest.yaml --environment production
```

Dry run (check file structure without applying):

```
monaco deploy manifest.yaml --dry-run
```

Validate Configurations

Monaco does **not** ship a standalone `monaco validate` subcommand. Validation runs as part of `monaco deploy --dry-run`, which loads the manifest, parses every project, checks that templates are valid JSON, and resolves cross-references — **without contacting the tenant**. It cannot validate the JSON payload against the schema, and it does not check reachability or token scopes: a deploy can still fail with HTTP 400 after a clean dry-run.

```
monaco deploy manifest.yaml --dry-run
```

Delete Configurations

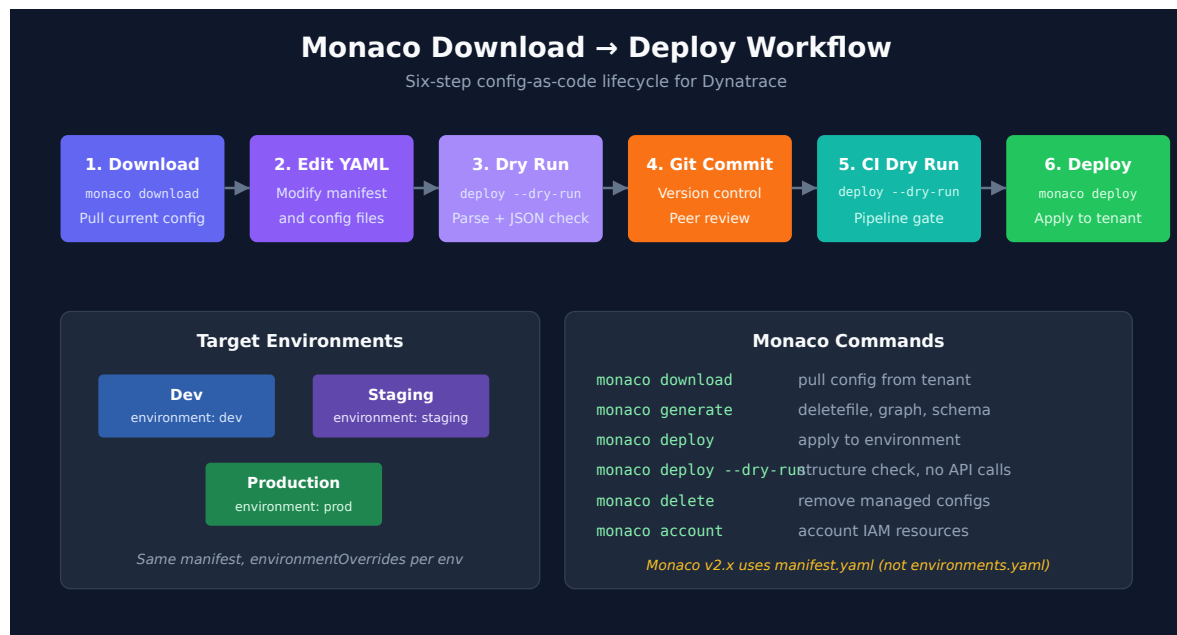
Remove configurations not in your project (orphans). The manifest is passed via `--manifest`, not as a positional argument:

```
monaco delete --manifest manifest.yaml --file delete.yaml
```

delete.yaml:

```
delete:
  - type: builtin:management-zones
    id: old-management-zone-id
```

Download vs Deploy Workflow



6. Advanced Features

Configuration Dependencies

Reference another configuration with a **reference parameter** — `[project, configType, configId, property]`. For Settings 2.0 configs the `configType` is the schema ID; `property: id` resolves to the referenced object's real Dynatrace ID at deploy time, and Monaco orders the deployment so the referenced object exists first:

```

configs:
  - id: my-alerting-profile
    config:
      name: Production Alerting
      template: alerting.json
      parameters:
        management_zone_id: ["my-project", "builtin:management-zones",
"production-mz", "id"]
      type:
        settings:
          schema: builtin:alerting.profile
          scope: environment

```

In `alerting.json`, use `{{ .management_zone_id }}`.

Conditional Configurations

`skip` is a boolean on `config:`. To deploy a config to only some environments, set a default and flip it per environment (or per environment group with `groupOverrides`):

```

configs:
  - id: production-only-config
    config:
      name: Production Detector
      template: detector.json
      skip: true
    type:
      settings:
        schema: builtin:davis.anomaly-detectors
        scope: environment
    environmentOverrides:
      - environment: production
        override:
          skip: false

```

Environment-Specific Overrides

`environmentOverrides` and `groupOverrides` sit at the config level, as siblings of `config:` — not inside it:

```

configs:
  - id: alerting
    config:
      name: Alerting Profile
      template: alerting.json
      parameters:
        delay_minutes: 5
    type:
      settings:
        schema: builtin:alerting.profile
        scope: environment
    environmentOverrides:
      - environment: production
        override:
          parameters:
            delay_minutes: 1

```

Grouping Configurations

Monaco v2 has no per-config `group:` key. Two different things are grouped, and neither is a config group:

- **Projects** group configurations. Put related configs in one project directory and deploy just that project with `--project`.
- **Environment groups** (`environmentGroups` in the manifest) group *target environments*. `--group` deploys to every environment in the named group; it does not select configs.

```

# Deploy only the web-application project, to every environment in the
"production" environment group
monaco deploy manifest.yaml --project web-application --group production

```

Best Practices

Practice	Description
Use meaningful IDs	IDs should describe the config purpose
Modular structure	Separate configs by domain (alerting, monitoring, etc.)

Practice	Description
Environment variables	Never hardcode tokens or URLs
Version control	Commit all changes with meaningful messages
Dry run first	Run <code>monaco deploy --dry-run</code> before applying — it catches YAML/JSON structure errors, not payload errors
Document parameters	Comment complex configurations
Check upgrade status before you invest	Before building a long-lived project around a schema, confirm it is not marked Blocked at upgrade in AUTOM-02's catalog — a blocked schema takes the whole Monaco config type with it

Common Issues and Solutions

Issue	Cause	Solution
"Config already exists"	Duplicate ID	Use unique IDs or download first
"Schema not found"	Typo in schema ID — or the schema was removed when the tenant upgraded to the latest Dynatrace	Check the exact schema name in the API. If it was correct and worked previously, check the schema's upgrade status in AUTOM-02's catalog before assuming a typo
"Template not found"	Wrong path	Use relative path from config.yaml
"Validation failed"	Invalid JSON	Check JSON syntax in template

7. Next Steps

Monaco in a CI/CD Pipeline (the most common next step)

Monaco itself is a CLI — to make it part of a GitOps flow, wire it into your CI/CD platform of choice. **AUTOM-07: CI/CD Integration** has the recipe per platform:

Platform	AUTOM-07 section
GitHub Actions	§3 GitHub Actions
GitLab CI/CD	§4 GitLab CI/CD
Bitbucket Pipelines	§5.1 Bitbucket Pipelines — Monaco Deploy
Atlassian Bamboo	§6 Atlassian Bamboo — adapt the Plan Specs YAML to call <code>monaco deploy</code>
Azure DevOps	§7 Azure DevOps Pipelines — adapt the Pipeline YAML to call <code>monaco deploy</code>

For the **full sequenced path** from zero to a working pipeline, see **AUTOM-01 §6 First-Time Setup Path — Path B (Monaco target)**.

When to Consider Adding Terraform

Scenario	Why Terraform wins
Cross-system orchestration (cloud + Dynatrace + Git in one apply)	Monaco is Dynatrace-only
State management + drift detection on critical resources	Monaco has no state file; drift detection happens via plan-reapply rather than state diffing
Part of larger IaC stack	Existing Terraform workflows extend naturally to Dynatrace
Lifecycle protections (<code>prevent_destroy</code> , <code>ignore_changes</code>)	Monaco has no per-resource lifecycle policy

For the framing of when a Terraform shop should *also* adopt Monaco (the reverse direction), see **AUTOM-01 §5: When a Terraform Shop Should Add Monaco**.

Continue the Series

Next Notebook	Focus
AUTOM-04: Terraform Provider	Infrastructure-as-code approach to Dynatrace configuration
AUTOM-07: CI/CD Integration	GitOps patterns for both Monaco and Terraform
AUTOM-08: Migration Automation	Tenant-to-tenant migration including <code>monaco download</code>

Additional Resources

- [Monaco commands reference \(DT docs\)](#) — *"A dry-run doesn't connect to Dynatrace and can't validate the content of the JSON sent to Dynatrace."*
- [Monaco configuration YAML reference \(DT docs\)](#) — parameters (`environment` , `reference`), `skip` , `environmentOverrides` / `groupOverrides`
- [Monaco manifest and resources \(DT docs\)](#) — *"the name value is the name of the environment variable that holds the secret, not the secret itself."*
- [Install Monaco \(DT docs\)](#)
- [Monaco Documentation](#)
- [Monaco Examples](#)
- [Configuration Schema Reference](#)

Summary

In this notebook, you learned:

- How to install and configure Monaco
- Project structure and manifest configuration
- Creating and deploying configurations via YAML
- Advanced features: reference parameters, `skip` , environment/group overrides, and project-scoped deploys

Key Takeaway: Monaco is ideal for teams wanting GitOps-style configuration management. It provides the right balance of simplicity and power for most Dynatrace automation needs. For the from-zero-to-pipeline sequence, see **AUTOM-01 §6** Path B.

Continue to AUTOM-04: Terraform Provider to learn infrastructure-as-code patterns, or jump to AUTOM-07: CI/CD Integration to wire Monaco into a pipeline now.

Community Resources & Examples

The following GitHub repositories provide starter templates, real-world examples, and hands-on exercises for Monaco:

Official Repositories

Repository	Description
dynatrace-configuration-as-code	Official Monaco CLI (v2.29.1 at time of writing, 09/2026 — check releases for newer) -- Go binary, Apache-2.0
dynatrace-configuration-as-code-samples	Official samples repo with 9 Monaco starter templates in <code>basic-templates-monaco</code>
easytrade	Demo microservices app with a working <code>monaco/</code> directory (manifest.yaml, detection rules, workflows)
Dynatrace-Config-Manager	GUI tool for tenant-to-tenant config migration; complements Monaco for brownfield scenarios

Starter Templates (in `dynatrace-configuration-as-code-samples`)

Template Directory	What It Configures
<code>basic-templates-monaco</code>	Alerting, app detection, synthetic, maintenance window, management zones, ownership, notifications, SLOs

Template Directory	What It Configures
<code>learn-monaco-auto-tag</code>	Auto-tagging with Monaco
<code>account-monaco-admin-access</code>	Admin access setup for Monaco

Pipeline Observability Samples

Monaco configurations for ingesting CI/CD pipeline events via OpenPipeline:

Directory	CI/CD Platform
<code>github_pipeline_observability</code>	GitHub Actions
<code>gitlab_pipeline_observability</code>	GitLab CI
<code>azure_devops_observability</code>	Azure DevOps
<code>argocd_observability</code>	ArgoCD

Training & Exercises

Repository	Description
monaco-self-paced-exercises	6 structured exercises: install, auto-tag, download, variables, delete/restore, linking configs
monaco-demo	Working GitHub Actions workflow for Monaco deploy with "crawl-walk-run" adoption methodology

Note: Monaco v1 (`dynatrace-oss/dynatrace-monitoring-as-code`) is no longer supported. Current Monaco releases have no conversion command: *"The last Monaco version to support the convert command is Monaco version 2.19.0."* Either run Monaco 2.19.0's `convert` per the [migrating to v2 guide \(DT docs\)](#), or re-run `monaco download` against the source tenant.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.